Fast Weights Predict, Not Evaluate: A Hebbian Associative World Model for Interception

Gabriel Hill
gabriel@familyfungroup.com

Abstract

The Dragon Hatchling (BDH) architecture is built on a fast-weight associative memory — a matrix of stimulus–response associations maintained by outer-product plasticity and read out by similarity-weighted recall. We ask what this primitive is *for*: is it a value function, an evaluative map from states to expected return, or a world model, a predictive map from states to next states? We argue for the latter and settle the question empirically in a controlled interception task, where a turn-rate-limited pursuer must catch a moving target, with ten seeds per cell and Welch significance tests. Three findings emerge. **(1)** Used as a value function, the memory collapses: a linear TD learner with the same associative readout reaches 0–22 catches — under 6% of the best analytic predictor in every environment — and backpropagated DQN, PPO, and continuous-action SAC all fall *below the no-prediction reflex on every environment.* **(2)** Used as a world model with greedy lead-pursuit planning, the same memory stays within 14% of the best analytic predictor in every environment, and reaches 93% of a hand-crafted circle-fitter on the curved prey that fitter specializes to. **(3)** On persistently curving prey — where intercepting a trajectory, not a point velocity, is what matters — the learned world model beats first- and second-order analytic extrapolation by 18% and 196% respectively, because rolling forward the learned dynamics is more accurate than a truncated Taylor series. The advantage is significant under a two-sided Welch test, survives a noise sweep (vanishing only where the motion becomes genuinely unpredictable), and a closed-loop imagination variant does not help — pointing to a robust effect. A formulation comparison then asks whether the *specific* Hebbian update rule matters: it does not make the model the accuracy optimum — a recursive-least-squares (RLS) world model matches or beats it on every stationary prey. We then close most of that gap with a *natural-gradient* (per-synapse, metaplastic) Hebbian rule that recovers RLS's advantage while staying $O(D)$ local, and show the residual is exactly the off-diagonal co-activity term — the price of optimality. Finally, on a new suite of nonstationary and adversarial evaders — prey that jink, and prey that *evolve their evasion in response to being caught* — the optimal stationary estimator collapses and the Hebbian rule beats it by up to nearly three-to-one, precisely because it keeps learning where RLS has stopped. We interpret the results as supporting a Dreamer-style division of labor: fast-weight memories are best understood as *predictive* substrates trained by dense self-supervised error, with sparse, outcome-driven planning on top, rather than as value-function approximators.

1 Introduction

A central question in biologically plausible machine learning is what a particular synaptic plasticity rule *computes*. The Dragon Hatchling (BDH) architecture [1] places at its core a fast-weight associative memory: a weight matrix updated by an outer product between a pre-synaptic activity vector and a post-synaptic signal, and queried by an inner product that recalls stored associations weighted by similarity. This is one of the oldest and most general ideas in connectionism — the linear associator, the Hopfield memory, the "fast-weight programmer" [2, 3]. But *what should the post-synaptic signal encode?* Two readings are possible, and they lead to very different systems:

- **The evaluative reading.** The memory stores state → *value* associations, forming a value function $V(s)$ or $Q(s,a)$ that is learned by bootstrapped temporal-difference (TD) updates and queried by a policy to select actions.

- **The predictive reading.** The memory stores state → *next-state* associations, forming a world model $\hat{s}_{t+1} = f(s_t)$ that is trained by self-supervised prediction error and rolled forward (imagined) to plan.

These readings are not cosmetic: they make opposite claims about the role of the same plasticity rule. The BDH paper's own framing — and the broader literature on reward-modulated Hebbian learning [6] — has tended toward the evaluative reading. We argue the predictive reading is the correct one, and we support the argument with a controlled experiment.

We choose **interception** (pursuit) as the test bed because it makes the two readings sharply distinguishable. Intercepting a moving target requires *predicting where the target will be*, not merely evaluating how good the current state is. It is also a domain with clean analytic baselines (first- and second-order extrapolation) against which a learned model can be judged, and with an oracle ceiling we can compute. Finally, interception is a canonical biologically relevant behavior, present from dragonflies to predators to human ball-players.

Our contributions are:

1. A **conceptual reframing** of fast-weight associative memories as world models (predictive substrates) rather than value functions, with a precise three-factor Hebbian instantiation.

2. A **seeded, multi-environment interception benchmark** with analytic, model-based, and model-free baselines, a reset-on-catch protocol that measures genuine interception rather than "lock-on," and a forecast-error metric that isolates prediction quality.

3. **Empirical evidence** that (a) model-free value learning — including a linear TD learner using the *same* associative memory, a backprop DQN, PPO, and continuous-action SAC — fails catastrophically on interception (falling *below the no-prediction reflex* in every environment), while (b) the same memory as a world model reaches 87–99% of the best analytic predictor per environment and (c) beats first- and second-order analytic extrapolation on curved prey, with a closed-loop imagination ablation and a noise sweep delimiting the claim.

We are explicit about scope: this is a controlled two-dimensional pursuit task whose purpose is to isolate one conceptual question. Section 6 discusses limitations and directions for future work.

2 Related Work

Fast-weight memories. The idea of using rapid weight changes as a form of short-term memory traces back to [2], and has been connected to linear attention ("transformers are secretly fast-weight programmers") [3]. The Dragon Hatchling [1] centers a fast-weight associative memory as the bridge between transformer-style attention and brain-like computation. Our work uses the core primitive — the outer-product weight update and similarity-weighted readout — and asks what it should represent.

Hebbian and three-factor learning. Classical Hebbian learning co-activates pre- and post-synaptic units ($\Delta W \propto yx^\top$). Three-factor rules gate this plasticity by a third, neuromodulatory signal (e.g., reward or prediction error), yielding biologically plausible and provably convergent variants [6]. The delta (Widrow–Hoff / LMS) rule $W \leftarrow W + \eta\, \delta\, x^\top$, where δ is the prediction error, is precisely such a three-factor rule [8]; we use its normalized form.

Model-based RL and world models. Learning a model of the environment and planning with it is a classical RL paradigm. Dreamer [7] popularized the split we instantiate: train a world model densely on prediction error, then plan (imagine) rollouts with it, while the policy is driven by sparse task reward. Our world model is the same idea at the scale of a single associative matrix rather than a deep recurrent network.

The deadly triad. [4] identify three properties whose combination can cause divergence in TD learning: function approximation, bootstrapping, and off-policy updates. Our linear-Q baseline is a textbook instance of all three, and its failure is a live illustration.

Pursuit and interception. Interception is a classic control/geometry problem with analytic solutions (lead pursuit, proportional navigation) [9, 10] and biological relevance [11]. We use it as a clean, interpretable test bed rather than contributing new pursuit algorithms.

3 Problem Setting

3.1 Environment

We use a two-dimensional toroidal world of size 1200×800 px with discrete-time dynamics at $dt = 0.05$ s. A single **prey** moves autonomously; a single **chaser** (pursuer) attempts to catch it. A catch occurs when the chaser comes within a capture radius of 48 px; a 0.5 s cooldown follows before the next catch can register.

- **Chaser.** Constant top speed $v_c = 175$ px/s. Its heading changes at a rate bounded by $\omega_{\max} = 3.6$ rad/s (a *turn-rate clamp*). It cannot decelerate.
- **Prey.** Speed 160 px/s (0.91× the chaser), so the chaser cannot win by raw speed — it must *intercept*. Six dynamics (Table 1, top):
  - `const-vel` — constant velocity (linear; first-order extrapolation is exact).
  - `circling` — constant turn rate $\dot{\theta} \in \pm 1.2$ rad/s drawn per episode, so the prey runs in persistent arcs/circles.
  - `ou-turn`, `ou-vel` — Ornstein–Uhlenbeck (mean-reverting, stochastic) turn and speed.
  - `jump` — piecewise-constant heading with random jumps.
  - `flee` — reactive avoidance: steers away from the chaser and speeds up when approached.

  A seventh variant, `circling-noisy`, augments `circling` with Ornstein–Uhlenbeck noise on the turn rate and is used only in the noise sweep (Section 5.5).

The first two are the scientifically decisive pair: `const-vel` is where first-order extrapolation is already Bayes-optimal (a control), and `circling` is where intercepting a *trajectory* — not a point velocity — is what separates predictors. The stochastic (`ou-*`, `jump`) and reactive (`flee`) prey test robustness and are where we document honest limits.

3.2 Protocol and metrics

A naive "catches per fixed horizon" metric is dominated by **lock-on**: once a faster chaser is within the capture radius it rarely loses the prey, so every competent agent saturates at the cooldown rate and prediction quality is invisible. We therefore use a **reset-on-catch** protocol: on each catch the prey is teleported to a uniformly random location 400–600 px from the chaser. Every catch is then a *fresh interception from distance*, and the catch count directly measures interception efficiency.

We report **catches per episode** (mean $\pm$ standard deviation over seeds) as the primary metric, and a **forecast error** — mean absolute error between a predictor's forecast of the prey's future position and its true position, evaluated at a fixed 1–2 s horizon — as a direct measure of prediction quality independent of the catch/steering loop.

3.3 The oracle ceiling

Interception has a clean ceiling. If the chaser knew the prey's future trajectory exactly and could steer optimally under the turn-rate clamp, it would achieve an optimal catch rate. In practice the relevant ceiling for our comparison is the **lead-pursuit ceiling** — the catch rate of a chaser that steers perfectly at the true future prey position — which the analytic baselines approximate from below. We do not claim the chaser reaches the true optimum; we claim it reaches the lead-pursuit regime, and that the world model does so *without* a hand-crafted dynamics model.

4 Method

4.1 The fast-weight associative memory

Let $s \in \mathbb{R}^d$ be a normalized sensory state and $y \in \mathbb{R}^2$ a two-dimensional target (a velocity, below). A fast-weight memory is a matrix $W \in \mathbb{R}^{2\times d}$ of associations, written and read as:

$$\text{read:} \quad \hat{y} = W\phi(s), \qquad \text{write:} \quad W \leftarrow W + \eta\, \underbrace{(y - \hat{y})}_{\delta}\, \phi(s)^\top \Big/ \big(1 + \|\phi(s)\|^2\big). \tag{1}$$

The write is a **three-factor (error-gated) Hebbian** rule: the outer product of a pre-synaptic activity $\phi(s)$ and a post-synaptic prediction error δ, normalized for stability (normalized LMS), with a small synaptic weight decay. The read is **content-addressable**: $\hat{y} = W\phi(s)$ is the similarity-weighted recall of all stored associations, so the memory *generalizes across states* rather than storing a lookup table. This is the BDH fast-weight primitive; nothing else is added.

The sensory features $\phi(s)$ are, by default, **linear**: $\phi(s) = [\,1,\ \Delta x/R,\ \Delta y/R,\ v_x/v_{\max},\ v_y/v_{\max}\,]$, where Δ is the wrapped relative position of the prey and v its velocity. We deliberately do *not* hand the model the prey's turn rate or acceleration; it must infer curvature from raw observations. (An optional random-Fourier expansion can be added to make the readout nonlinear; Section 5.4 shows it is unnecessary here.)

4.2 The world model

We use the memory as a **world model**: it learns the prey's one-step transition $f : s_t \mapsto v_{t+1}$, mapping the current state to the *next* velocity. Training is dense and self-supervised — every observed tick updates the memory toward the true next velocity — with no reward signal. To predict the prey's position τ steps ahead, the model is **rolled forward (imagined)**: starting from the current state, it repeatedly predicts the next velocity and integrates it to a new position, for τ steps.

4.3 The planner

Planning is deliberately minimal and shared across all predictors: **lead pursuit** — steer at the predicted future prey position. The lead horizon is $\tau = d/v_c$ (the time to reach the prey at current speed), capped at 2 s, so the chaser aims at where the prey *will be*, not where it is. The chaser's heading is then moved toward that aim under the turn-rate clamp. The policy (which way to steer) is thus driven only by the forecast; the only thing that varies across predictors is *how the forecast is made*.

4.4 The contrast: the same memory as a value function

To make the value-vs-world-model contrast clean, we implement the evaluative reading with the *identical* substrate: $Q(s, a) = w_a \cdot \phi(s)$ for 8 discrete headings, trained by TD(0) [5] with ϵ-greedy exploration (Section 5). This is exactly a linear value function over the same features ϕ, updated by a bootstrapped, off-policy rule — the textbook "deadly triad" configuration [4]. Any performance gap between it and the world model is therefore attributable to the *role* of the memory (evaluate vs. predict), not to the substrate, features, or representational capacity.

5 Experiments

5.1 Setup

All results are seeded and reproducible; every cell is a mean over 10 seeds with standard deviation. Hyperparameters were not tuned per-environment except where an ablation explicitly varies them. The BDH world model uses the linear feature set (Section 4), learning rate $\eta = 0.5$, and weight decay $\lambda = 10^{-3}$.

Baselines.

- *Predictors* (lead pursuit, differing only in the forecast):
  - `velocity-lead`: $\hat{p} = p + v\tau$ (first-order extrapolation).
  - `accel-lead`: $\hat{p} = p + v\tau + \frac{1}{2}a\tau^2$ (second-order Taylor, a from a finite-difference estimate).
  - `kalman-lead`: a constant-velocity Kalman filter + lead.
  - `circle-fit`: a least-squares circle fitted to the recent trajectory (Kasa normal equations), then extrapolated along the fitted circle — an analytic *specialist* with the exact circular inductive bias.
  - `bdh`: the fast-weight world model (Section 4), rolled forward with the chaser held fixed (open loop).
  - `bdh-cl`: the same world model rolled forward while the chaser is also imagined to pursue (closed loop).
- *Policies* (learn or select the heading directly):
  - `pure-pursuit`: steer at the *current* prey position (the no-prediction reflex).
  - `mpc`: model-predictive control — search 8 headings, simulate a short horizon with a first-order (constant-velocity) prey model, minimize final distance.
  - `linear-q`: linear TD Q-learning over the same features ϕ (the evaluative reading).
  - `dqn`: an MLP Q-network with replay buffer and target network.

Table 1: Catches per episode (mean $\pm$ sd), reset-on-catch, 10 seeds. *Top:* lead-pursuit predictors ($\times$ 24,000 steps). *Bottom:* policies ($\times$ 20,000 steps).

prey	velocity-lead	accel-lead	kalman-lead	circle-fit	**bdh**	bdh-cl
const-vel	398 ± 51	398 ± 51	148 ± 52	284 ± 46	$\mathbf{397 \pm 49}$	396 ± 48
circling	327 ± 35	130 ± 192	35 ± 66	415 ± 15	$\mathbf{385 \pm 41}$	386 ± 38
ou-turn	368 ± 17	364 ± 10	116 ± 7	373 ± 11	$\mathbf{331 \pm 15}$	327 ± 17
ou-vel	398 ± 14	389 ± 10	140 ± 9	398 ± 15	$\mathbf{372 \pm 13}$	362 ± 17
jump	342 ± 10	293 ± 8	133 ± 10	282 ± 6	$\mathbf{296 \pm 13}$	275 ± 16
flee	195 ± 3	71 ± 10	9 ± 5	178 ± 2	$\mathbf{178 \pm 3}$	160 ± 7

prey	pure-pursuit	mpc	linear-q	dqn	ppo	sac
const-vel	80 ± 6	304 ± 37	18 ± 8	19 ± 15	30 ± 10	23 ± 13
circling	176 ± 67	256 ± 31	18 ± 3	23 ± 8	22 ± 6	27 ± 13
ou-turn	134 ± 10	269 ± 21	22 ± 5	19 ± 5	21 ± 3	25 ± 9
ou-vel	144 ± 9	308 ± 14	18 ± 5	21 ± 6	25 ± 7	28 ± 5
jump	252 ± 10	261 ± 8	20 ± 5	25 ± 4	24 ± 5	28 ± 7
flee	170 ± 3	152 ± 2	0 ± 0	3 ± 3	0 ± 0	7 ± 5

  – `ppo`: clipped-surrogate PPO with policy and value MLPs.
  – `sac`: Soft Actor-Critic with a continuous turn-rate action (reparameterized actor, twin critics, fixed entropy coefficient $\alpha = 0.1$).

The model-free learners receive a dense reward $r = -\text{distance}/600$ plus a +1 catch bonus, so their failure is a credit-assignment/interception failure rather than an artifact of sparse reward. Section 5.6 additionally varies the *update rule* of the world model itself (plain LMS, recursive least squares, and an MLP) to ask whether the fast-weight formulation is necessary; Section 5.7 improves that rule with a natural-gradient (metaplastic) learning rate, and Section 5.8 tests it on a suite of nonstationary and adversarial (co-evolving) prey.

5.2 Result 1: world model vs. value function

Table 1 (bottom) reports catches under the reset-on-catch protocol. The contrast is stark and consistent across all six environments:

- The **linear-Q learner — the same associative memory used as a value function — collapses** to 0–22 catches, under 6% of the best analytic predictor in each environment.
- The **backprop DQN, PPO, and continuous-action SAC all fall *below the no-prediction reflex*** (pure pursuit) in *every* environment: they do worse than a policy that does no learning and no prediction at all. This failure is not an artifact of discrete actions, since SAC's action space is continuous.
- The **BDH world model**, by contrast, reaches 178–397 catches — within 14% of the best analytic predictor per environment — with no value function and a fraction of the training.

Because linear-Q and the world model use the *same* features and the *same* associative substrate, the gap is clean evidence that the fast-weight memory succeeds when it *predicts* and fails when it *evaluates*.

The world model's advantage is specific and predictable: it *ties* first-order on linear motion (`const-vel`, as it should for a linear model), *beats* first-order decisively on curved motion (`circling`, +18%), and falls *slightly below* first-order on the stochastic and reactive prey (`ou-*`, `jump`, `flee`) where the future is only partially predictable from the current state and the learned model adds variance. Across all six environments the world model stays within 14% of the *best* analytic predictor for that environment. By contrast, model-free value learning (linear-Q, DQN, PPO, SAC) is *below the reflex* everywhere, and MPC — a planner with a first-order model — decisively beats all of them. Prediction matters, and a learned predictive model is competitive with the analytic frontier exactly where the motion is smooth and curved.

5.3 Result 2: world model vs. analytic extrapolation on curved prey

On `circling` (Table 1, top), the BDH world model achieves 385 ± 41 catches, +18% over first-order extrapolation (327 ± 35) and +196% over second-order (130 ± 192). Both advantages are significant under a two-sided Welch test ($t = 3.43$, $p = 0.0031$ vs. first-order; $t = 4.10$, $p = 0.0022$ vs. second-order). The second-order baseline is also wildly unreliable — its standard deviation (192) is larger than its mean — whereas the world model is stable (± 41).

The mechanism is visible in the forecast error. On `circling`, the world model's lead-horizon forecast error is 58 ± 13 px versus 96 ± 51 px for first-order extrapolation: rolling forward the learned rotation tracks the arc, while a straight-line extrapolation misses it. The second-order baseline is a cautionary tale — its *mean* forecast error (54 px) is deceptively low, but its variance (± 43 px) is enormous: the $\frac{1}{2}a\tau^2$ term of a second-order Taylor expansion *overshoots* the circle over the ~ 2 s lead horizon, and a single bad forecast during the final approach costs the catch. A learned model that re-applies the true (learned) one-step dynamics at each of τ steps is the correct object, and the fast-weight memory discovers it from raw observations without being told the prey is circling.

The circle-fit specialist is the one analytic baseline that edges out the world model on `circling` (415 ± 15 vs. 385 ± 41, $p = 0.048$): its forecast error is 11 ± 6 px, near-perfect, because it is handed the exact circular inductive bias. It also transfers to the smooth-turning OU prey (`ou-turn` 373, `ou-vel` 398) but collapses on straight (`const-vel` 284) and discontinuous (`jump` 282) motion, where a circle is the wrong object. The world model, by contrast, is a *generalist*: it reaches 93% of the specialist on `circling` and beats the specialist on `const-vel` (397 vs. 284) and `jump` (296 vs. 282), and ties it on `flee` (178 vs. 178).

On `const-vel` the world model ties first-order (as it should: the dynamics are linear, and a linear model reproduces them exactly), confirming the model adds no spurious advantage. On the stochastic prey (`ou-*`, `jump`) no predictor beats first-order — the future is genuinely unpredictable from the current state — and the learned model adds a small amount of variance. These are the honest negative results that delimit the claim.

5.4 Ablations

We verify the result is not fragile. (All ablations on `circling`, reset-on-catch, 10 seeds.)

- **Turn-rate clamp** ($1.8 \to 5.4$ rad/s): BDH beats first-order at *every* turn rate (329/371/385/395 vs. 261/315/327/336), so the advantage is not an artifact of the clamp's value.
- **Speed ratio** (prey 140/150/160 px/s vs. chaser 175): BDH is robust (411/401/385). Second-order only competes at slow prey (416 at 140) and collapses at 160 (130) as its parabola overshoots a fast circle.

Table 2: Welch t-tests (two-sided, 10 seeds per cell).

comparison	prey	t	p
BDH vs velocity-lead	circling	3.43	0.0031
BDH vs accel-lead	circling	4.10	0.0022
BDH-cl vs BDH	flee	−8.07	4.6×10^{-6}
circle-fit vs BDH	circling	2.21	0.048
BDH vs SGD (LMS)	circling	0.43	0.68
BDH vs RLS	circling	−2.11	0.053
BDH vs MLP	circling	14.92	2.2×10^{-9}
BDH vs RLS	flee	20.86	2.4×10^{-9}
BDH vs RLS	jump	−10.59	1.0×10^{-8}
BDH vs RLS	ou-turn	−6.18	8.9×10^{-6}
BDH vs RLS	ou-vel	−4.76	1.6×10^{-4}
BDH vs SGD (LMS)	flee	11.39	3.7×10^{-9}
SAC vs reflex	circling	−6.90	5.1×10^{-5}
DQN vs reflex	circling	−7.17	4.6×10^{-5}

- **Memory width (Fourier features M)** (0/8/24/48): the *linear* associator ($M = 0$) is optimal (385); the nonlinear expansion only adds variance (308–369). The relevant dynamics here are smooth, and a linear content-addressable memory captures them.
- **Learning rate η** (0.1/0.3/0.5/1.0): the model is insensitive to η (382–387); *every* η beats first-order (327).
- **Weight decay λ** ($0/10^{-4}/10^{-3}/10^{-2}$): moderate decay is best ($10^{-3} \to 385$); every λ beats first-order.
- **Action discretization** (4/8/16 headings, model-free): linear-Q (18–28), DQN (21–23), and PPO (22–24) stay far below the reflex (176) at every discretization, so the model-free failure is *not* a discretization artifact — it persists at 16 actions.

5.5 Robustness: significance, noise, and closed-loop imagination

Significance. Table 2 reports two-sided Welch tests on the key comparisons. The world model's advantage over both analytic extrapolators on `circling` is significant ($p \leq 0.0031$); the circle-fit specialist's edge over the world model on `circling` is significant but small ($p = 0.048$); the model-free learners' collapse below the reflex is highly significant ($p < 10^{-4}$); and the closed-loop variant's *worsening* on `flee` is highly significant ($p = 4.6 \times 10^{-6}$). The formulation comparisons (Section 5.6) are equally decisive: the optimal RLS estimator beats BDH on `jump` ($p = 1.0 \times 10^{-8}$) and on `ou-turn`/`ou-vel`, while BDH beats RLS on `flee` ($p = 2.4 \times 10^{-9}$); plain LMS is indistinguishable from BDH on `circling` ($p = 0.68$), and the MLP is decisively worse ($p = 2.2 \times 10^{-9}$).

Noise sweep. To locate the boundary of "predictable enough," we add Ornstein–Uhlenbeck noise of increasing scale to the `circling` turn rate (`circling-noisy`) and compare the world model with first-order extrapolation (Table 3). The world model's advantage is large and robust across noise scales 0–0.6 (+56 to +70 catches) and collapses only at the extreme scale 1.2 (−1), where the injected noise dominates the underlying circle. This sharpens limitation 1 of Section 6.3: the world model wins exactly where the signal (the circle) is recoverable from the noise.

Closed-loop imagination. The open-loop world model holds the chaser fixed while rolling the prey forward, a crude model of reactive (`flee`) prey that respond to the chaser. We therefore also

Table 3: Noise sweep on `circling-noisy` (10 seeds $\times$ 24,000).

PREY_NOISE	velocity-lead	bdh	bdh $-$ vel
0.0	323 ± 34	379 ± 48	$+56$
0.15	320 ± 38	389 ± 39	$+69$
0.3	323 ± 35	393 ± 28	$+70$
0.6	321 ± 36	380 ± 28	$+59$
1.2	328 ± 21	326 ± 25	-1

evaluate a closed-loop variant (`bdh-cl`) that, during the rollout, imagines the chaser steering toward the predicted prey position and moves it accordingly. Counter to the hope that this would close the gap on `flee`, it is *significantly worse* (160 ± 7 vs. 178 ± 3, $p = 4.6 \times 10^{-6}$) and no better elsewhere (Table 1). This is a negative result: as implemented, closed-loop imagination does not transfer to reactive prey, and we leave modeling the pursuit–evasion interaction as future work (Section 6.4).

5.6 Result 3: is the fast-weight rule necessary?

The results so far show that a world model *helps*, but not whether the *specific* Dragon-Hatchling update rule — normalized error-gated Hebbian plasticity with weight decay — is what does the work, or whether any online learner would. We re-run the predictor comparison with the world model's update rule replaced by three alternatives, holding the feature space, the τ-step rollout, and the lead-pursuit planner fixed:

- `sgd` (LMS): plain online least-mean-squares, $W \leftarrow W + \eta\,(y - \hat{y})\,\phi^\top$, with no normalization, no error gating, and no decay [8].
- `rls`: recursive least squares — the optimal online *linear* estimator — maintaining a full $D \times D$ covariance matrix with exponential forgetting ($\lambda = 0.999$) [13].
- `mlp`: a one-hidden-layer (16-unit ReLU) network trained online with Adam [14], the nonlinear analog of the same mapping.

Table 4 reports catches. Three findings follow.

1. **The optimal classical estimator beats the Hebbian rule on stationary prey.** RLS ties BDH on `const-vel` (398 vs. 397) and beats it on every other stationary prey: `circling` (416 ± 24 vs. 385 ± 41, $p = 0.053$), `ou-turn` (371 vs. 331, $p = 8.9 \times 10^{-6}$), `ou-vel` (399 vs. 372, $p = 1.6 \times 10^{-4}$), and `jump` (349 vs. 296, $p = 1.0 \times 10^{-8}$). In raw predictive accuracy the fast-weight rule is therefore *not* the optimum: a full-covariance least-squares estimator is. Notably, RLS also reaches the circle-fit ceiling on `circling` (416 vs. 415), confirming that the world model's residual gap there is a *learning-efficiency* gap, not a representational one — the circular dynamics are linear in the velocity features (a fixed rotation), which the optimal estimator fits near-perfectly.
2. **The Hebbian rule's distinctive edge is adaptivity, not asymptotic accuracy.** On the *reactive* prey `flee`, the ranking reverses: BDH beats RLS by more than two-to-one (178 ± 3 vs. 86 ± 14, $p = 2.4 \times 10^{-9}$). The error-gated three-factor update with weight decay tracks the rapidly changing prey-to-velocity map far better than a slow-forgetting covariance estimator, which has effectively stopped adapting by the time the prey's evasion policy has changed.

Table 4: World-model formulation (mean ± sd catches, reset-on-catch, 10 seeds × 24,000).

prey	bdh (Dragon Hatchling)	sgd (LMS)	rls	mlp
const-vel	397 ± 49	398 ± 51	398 ± 50	297 ± 128
circling	385 ± 41	374 ± 71	$\mathbf{416 \pm 24}$	174 ± 19
ou-turn	331 ± 15	336 ± 8	$\mathbf{371 \pm 13}$	222 ± 27
ou-vel	372 ± 13	373 ± 17	$\mathbf{399 \pm 13}$	258 ± 33
jump	296 ± 13	296 ± 13	$\mathbf{349 \pm 9}$	254 ± 13
flee	$\mathbf{178 \pm 3}$	162 ± 4	86 ± 14	161 ± 8

3. **The specific Hebbian ingredients matter little on stationary prey, and nonlinearity actively hurts.** Plain LMS is statistically indistinguishable from BDH on `circling` (374 ± 71 vs. 385 ± 41, $p = 0.68$) — the normalization and decay do not produce the world-model advantage there — yet on `flee` BDH beats LMS too (178 vs. 162, $p = 3.7 \times 10^{-9}$), so the gating/decay contributes specifically where the map is nonstationary. The MLP is the *worst* formulation on every stationary prey (174 vs. 385 on `circling`, $p = 2.2 \times 10^{-9}$), reinforcing the ablation of Section 5.4: a *linear* content-addressable memory is the right model class for smooth dynamics, and nonlinearity only adds variance.

Interpretation. "Dragon Hatchling" is best read as the *plausible* world-model formulation: it is the only one realizable by local, outer-product, three-factor synaptic plasticity — no $O(D^2)$ covariance matrix, no matrix inversion, no backpropagation — and it remains *competitive* with the optimal estimator on stationary prey (within 8% on `circling`) while being *uniquely* robust to nonstationarity (more than two-to-one on `flee`). It is not the accuracy champion; it is the formulation that trades a small asymptotic gap for locality and adaptivity. The headline contrast of Section 5 — world model versus value function — is unaffected: RLS and BDH are both *world models*, and both dominate the evaluative reading of the same memory.

5.7 Result 4: a natural-gradient Hebbian rule nearly matches the optimal estimator

Section 5.6 leaves open a constructive question: RLS beats the Hebbian rule on stationary prey, but *why*, and how much of that gap can a local rule recover? The diagnosis is a curvature effect, not a representational one. RLS's per-step gain is $P_t\phi_t \approx (\mathbb{E}[\phi\phi^\top])^{-1}\phi_t$ — the inverse feature covariance applied to the feature — whereas BDH's NLMS gain is the *scalar* step $\eta\,\phi_t/(1 + \|\phi_t\|^2)$. The inverse covariance decomposes into a **diagonal** part (per-synapse second moments) and an **off-diagonal** part (co-activity between synapses). We therefore modify the Hebbian rule by replacing the scalar gain with a per-synapse, natural-gradient gain: each synapse i keeps a running second moment

$$g_{t,i} \leftarrow (1-\beta)\, g_{t-1,i} + \beta\, \phi_{t,i}^2, \qquad W \leftarrow W + \eta\,(y - \hat{y})\,\big(\phi_t \oslash (\varepsilon + \sqrt{g_t})\big)^\top, \tag{2}$$

where $\oslash$ is elementwise division and $\varepsilon > 0$. This is the *diagonal* natural gradient — AdaGrad's per-parameter preconditioner [15], read as metaplastic per-synapse learning rates — and the update remains local, outer-product, and three-factor. As a second candidate we add a slow Polyak–Ruppert-averaged readout, $W_s \leftarrow (1-\kappa)W_s + \kappa W$ [16], the classical device for making a first-order rule asymptotically optimal.

Table 5 reports catches for the two ingredients separately and together (`bdh-pre`: preconditioning only; `bdh-avg`: averaging only; `bdh-ng`: both). Three findings follow.

Table 5: Natural-gradient Hebbian rule (mean ± sd catches, reset-on-catch, 10 seeds × 24,000).

prey	bdh	rls	bdh-pre (natural gradient)	bdh-avg (averaging)	bdh-ng (both)
const-vel	397 ± 49	$\mathbf{398 \pm 50}$	398 ± 51	398 ± 51	397 ± 51
circling	385 ± 41	$\mathbf{416 \pm 24}$	408 ± 34	363 ± 88	394 ± 47
ou-turn	331 ± 15	$\mathbf{371 \pm 13}$	352 ± 15	310 ± 13	307 ± 13
ou-vel	372 ± 13	$\mathbf{399 \pm 13}$	381 ± 17	341 ± 11	348 ± 9
jump	296 ± 13	$\mathbf{349 \pm 9}$	301 ± 6	312 ± 18	294 ± 10
flee	$\mathbf{178 \pm 3}$	86 ± 14	173 ± 5	138 ± 5	162 ± 2

1. **The diagonal part recovers RLS's edge where the gap is curvature.** `bdh-pre` statistically *matches* RLS on `const-vel` (398 ± 51 vs. 398 ± 50, $p = 1.0$) and on `circling` (408 ± 34 vs. 416 ± 24, $p = 0.55$) — the latter eliminating the marginal-but-real gap BDH showed there (385 vs. 416, $p = 0.053$) — and significantly narrows `ou-turn` (352 vs. 371, $p = 0.007$) and `ou-vel` (381 vs. 399, $p = 0.017$), while — critically — *keeping* BDH's reactive-prey edge intact (`flee` 173 vs. 86, $p = 8.8 \times 10^{-10}$). The per-synapse normalization supplies the diagonal of the covariance RLS inverts, at $O(D)$ cost.

2. **The residual has two distinct causes.** On `circling` the remaining gap (408 vs. 416) is the *off-diagonal* v_x–v_y co-activity that a diagonal preconditioner cannot see — exactly the term RLS adds at the cost of an $O(D^2)$ covariance and a matrix inversion — and RLS is already at the circle-fit ceiling there (416 vs. 415, Section 5.6), so no predictor can beat it. On `jump` the gap (301 vs. 349, $p = 2.7 \times 10^{-10}$) is instead a *noise-averaging* gap: jump teleports inject velocity spikes, and RLS's covariance smooths them while a constant-gain rule over-reacts — a different deficit, which the preconditioner does not address.

3. **Polyak–Ruppert averaging is a net negative here.** `bdh-ng` (both ingredients) is *worse* than `bdh-pre` (preconditioning only) on `ou-turn` (307 vs. 352), `ou-vel` (348 vs. 381), and `flee` (162 vs. 173), because the averaged readout lags the fast weight on a closed-loop reactive map, and constant-step LMS's $O(\eta)$ steady-state bias is not removed by averaging the iterates. The natural-gradient preconditioner alone is the effective improvement.

Interpretation. The improved rule — per-synapse natural-gradient (metaplastic) Hebbian plasticity — recovers most of the optimal estimator's advantage on stationary prey while preserving the reactive-prey win, from $O(D)$ local, outer-product, three-factor updates alone. The remaining gap is precisely the off-diagonal co-activity term, the $O(D^2)$ curvature that is the price of optimality; on the one prey where that gap is largest, RLS is already at the analytic ceiling. The frontier is therefore sharp: a local Hebbian rule can be made *near-optimal* and *uniquely adaptive*, but it cannot be both fully optimal and fully local — optimality on curved, correlated dynamics buys exactly one thing, the inverse covariance, and that is the one thing locality forbids.

5.8 Result 5: the adaptive edge — Hebbian memory wins on nonstationary and adversarial prey

Every result so far is on prey whose dynamics are stationary or only weakly reactive. There, RLS is optimal and the Hebbian rule is merely competitive. But the motivating claim for a fast-weight memory is *adaptivity*: a rule that writes on every sample should track a world that *changes*. We therefore add a nonstationary suite of reactive evaders — prey whose state → velocity map is not fixed, but is driven by, and co-evolves with, the pursuer itself:

Table 6: Nonstationary and adversarial prey (mean $\pm$ sd catches, reset-on-catch, 10 seeds $\times$ 24,000).

prey	velocity-lead	circle-fit	bdh	bdh-cl	rls	bdh-ng
flee	195 ± 3	178 ± 2	178 ± 3	160 ± 7	86 ± 14	162 ± 2
zigflee	227 ± 5	216 ± 4	208 ± 5	206 ± 4	138 ± 8	187 ± 5
adversarial	161 ± 2	144 ± 1	146 ± 3	129 ± 9	51 ± 9	139 ± 3

- `zigflee`: flees the chaser but periodically re-samples a random heading jink, so the reactive map changes throughout the episode.
- `adversarial`: an "evolve-as-you-evolve" evader. It flees, but a persistent internal *evasiveness* rises each time it is caught and decays while it escapes, so the harder the chaser presses, the more evasive the prey becomes — the prey's policy adapts to the pursuer's own success, with the sole goal of not being caught.

Table 6 reports catches on the nonstationary suite. Three findings follow.

1. **The optimal stationary estimator collapses; the Hebbian rule does not.** On all three reactive prey, RLS — the estimator that wins on stationary prey — is the worst world model. BDH beats it on `flee` (178 vs. 86, $p = 2.4 \times 10^{-9}$), on `zigflee` (208 vs. 138, $p = 6.4 \times 10^{-13}$), and most decisively on `adversarial` (146 vs. 51, $p = 2.5 \times 10^{-12}$). On `adversarial`, RLS falls *below the no-prediction reflex* (51 vs. 141): the rule that "gives up" adapting when its covariance has converged is exactly the rule that fails when the world refuses to sit still.
2. **The co-evolving prey is where adaptivity matters most.** The largest margin is on `adversarial` (nearly three-to-one, 146 vs. 51), where the map changes *as a direct consequence of the pursuer's own learning*: every catch the chaser earns is answered by an escalation of evasion, and only a rule that keeps learning tracks the new map. This is the regime the fast-weight primitive is for.
3. **The natural-gradient variant inherits the edge.** `bdh-ng` decisively beats RLS across the suite (`flee` 162, `zigflee` 187, `adversarial` 139, all $p < 10^{-7}$), because what does the tracking is BDH's fast weight, which the natural-gradient refinement leaves intact. The locality–optimality trade-off of Section 5.7 is therefore resolved in favor of locality exactly where the world is not stationary.

Interpretation. Together with Section 5.7, the picture is complete and honest. RLS is the right world model when the world is a stationary linear program — the regime where an $O(D^2)$ covariance fit is optimal and its forgetting is cheap. The Hebbian rule is the right world model when the world is nonstationary, reactive, or adversarial — the regime of living prey that adapt to being pursued. Section 5.6's caveat and this section's advantage are two sides of the same trade-off: the inverse covariance buys stationary accuracy and sells adaptivity; the three-factor Hebbian update buys adaptivity and sells only the off-diagonal curvature that stationary optimality requires.

6 Discussion

6.1 Why model-free value learning fails here

Three compounding factors explain the model-free collapse. First, the **deadly triad** (linear-Q is linear function approximation + TD bootstrapping + ϵ-greedy off-policy updates [4]) makes the value estimate unstable or divergent. Second, the **turn-rate clamp** means the greedy action cannot be executed instantaneously: even a perfect value function's argmax heading takes $\sim$0.1 s to reach, so one-step-greedy value learning systematically under-turns. Third, the **interception reward is inherently temporal**: catching a moving target requires anticipating a future position, which is a prediction problem, not a credit-assignment problem. A value function must propagate reward through many time steps of a high-frequency control loop to *implicitly* represent the same thing a world model *explicitly* predicts in one step.

The dense reward and strong function approximators (DQN, PPO, SAC) do not rescue this: they reliably learn to *approach* the prey but not to *intercept* it, and frequently do worse than the reflex.

6.2 Why the world model works

Interception is a prediction problem. The world model's inductive bias — learn the map $s_t \mapsto v_{t+1}$ and roll it forward — is exactly aligned with the task, and its training signal (dense, self-supervised prediction error) is abundant at every tick, independent of the sparse catch reward. This is the Dreamer split [7] in miniature: the policy (steer at the forecast) is driven by outcome, while the model is driven by prediction. The fact that a *linear* associative memory suffices here suggests that, for smooth physical dynamics, content-addressable recall of local transitions is already a strong world model — and that the "fast weights" of the BDH architecture are best read as a predictive substrate. Section 5.6 sharpens this: the world-model advantage is a property of *predicting*, not of the specific Hebbian rule, since the optimal RLS estimator matches or exceeds the Hebbian model on stationary prey. Sections 5.7 and 5.8 complete the picture. A natural-gradient (per-synapse, metaplastic) Hebbian rule recovers most of RLS's stationary edge while staying $O(D)$ local (Section 5.7), and on nonstationary, adversarial prey — where the map the model must learn is itself driven by the pursuer's own success — the Hebbian rule beats the optimal estimator by up to nearly three-to-one, because its constant-gain three-factor update keeps learning where RLS's converged covariance has stopped (Section 5.8). Locality and adaptivity, not asymptotic optimality, are what the three-factor rule buys — and adaptivity is what real, evading prey demand.

6.3 Limitations

This is a controlled single-task study; we do not claim generality to other domains. Specifically:

1. The result is strongest where it should be — curved, predictable motion — and does not hold on genuinely unpredictable (Martingale) prey, where no predictor beats first-order. The noise sweep (Section 5.5) makes this boundary explicit: the advantage vanishes only where the motion becomes unpredictable.

2. On **reactive** prey (`flee`), the world model is only comparable to first-order. A closed-loop imagination variant that also simulates the chaser's pursuit during the rollout does *not* help — it is significantly worse (160 ± 7 vs. 178 ± 3, $p = 4.6 \times 10^{-6}$), so reactive evasion remains an open gap rather than a solved one.

3. The world model is **linear**; the nonlinear (Fourier) expansion did not help on these smooth dynamics, so we do not demonstrate nonlinear world modeling.

4. On the smooth-turning prey, the hand-crafted circle-fit specialist beats the world model; the world model is a generalist that approaches but does not exceed the analytic ceiling on the prey that analytic model is designed for.

5. The fast-weight update rule is **not the accuracy optimum on stationary prey**: the optimal RLS estimator matches or beats it there (Section 5.6). A natural-gradient (per-synapse, metaplastic) refinement recovers most of that gap, and the residual is the off-diagonal co-activity term — the $O(D^2)$ curvature that is the price of optimality (Section 5.7). The Hebbian rule's case therefore rests on adaptivity and locality: it matches near-optimal accuracy on stationary prey and *beats* the optimal estimator on nonstationary, adversarial prey (Section 5.8), not on being the raw one-step optimum.

6. The environment is single-prey, toroidal, and two-dimensional; real pursuit adds walls, multiple prey, partial observation, and sensing noise, and the model-free baselines (DQN, PPO, SAC) are not hyperparameter-tuned, so their failure is a lower bound on what tuned methods could achieve.

6.4 Future work

With ten seeds, significance tests, a continuous-action baseline, an analytic circle-fit specialist, a noise sweep, a closed-loop ablation, a world-model formulation comparison, a natural-gradient refinement, and a nonstationary/adversarial suite in place, the remaining gaps are about *generality*, not about the core contrast. Specifically: (i) **reactive prey** — the Hebbian rule decisively beats the optimal estimator on reactive prey (Section 5.8), but no predictor yet beats the no-learning first-order baseline there; modeling the pursuer–evader interaction remains open. (ii) **standard benchmarks** — reproducing the value-vs-world-model contrast on a standard interception or control suite, rather than this purpose-built task, would test its generality. (iii) **nonlinear world modeling** — the linear associator sufficed for these smooth dynamics; domains with genuinely nonlinear transitions are where a richer readout (Fourier features, or a deep successor) should be required. (iv) **tuned model-free baselines** — DQN, PPO, and SAC were not exhaustively hyperparameter-tuned, so their collapse here is a documented failure of the *interception* objective under default settings, not a claim that no RL method can ever intercept.

7 Conclusion

We asked whether the fast-weight associative memory at the heart of the Dragon Hatchling architecture is a value function or a world model. In a seeded interception benchmark with ten seeds and significance tests, the answer is decisive: the *same* memory, used to predict, stays within 14% of the best analytic predictor per environment and beats first- and second-order analytic extrapolation on curved prey, while used to evaluate it collapses to 0–22 catches — and even strong model-free learners (DQN, PPO, SAC) fall below a no-prediction reflex in every environment. The result is robust across a noise sweep and a closed-loop ablation, and is bounded honestly: on unpredictable motion no predictor beats first-order, and on smooth curved motion a hand-crafted circle-fitter remains the specialist ceiling that the general-purpose world model approaches from below. A formulation comparison adds the honest caveat that the *specific* Hebbian rule is not the accuracy optimum — a recursive-least-squares world model matches or beats it on stationary prey. We then close most of that gap with a natural-gradient (per-synapse, metaplastic) Hebbian rule, showing the residual is exactly the off-diagonal co-activity term that is the price of optimality, and we show that on nonstationary, adversarial prey — evaders that jink, or that *evolve their evasion in response to being caught* — the Hebbian rule beats the optimal estimator by up to nearly three-to-one, because it keeps learning where the optimal estimator has stopped. The result supports a clean reading of fast-weight memories as predictive substrates trained by dense self-supervised error, with sparse outcome-driven planning on top — a Dreamer-style division of labor instantiated by a biologically plausible three-factor Hebbian rule. We hope this reframing is useful both to readers of the BDH literature and to practitioners choosing what to put in a fast-weight memory: put the *next state*, not the *value*.

Acknowledgments and disclosures

Author responsibilities. All authors take full responsibility for the content of this manuscript, regardless of how it was produced.

Use of generative AI language tools. Portions of this manuscript were drafted and edited with the assistance of a text-to-text generative AI language model. All results, code, references, and claims were reviewed and verified by the authors, who accept full responsibility for them; the AI tool is not an author. This statement is included to comply with arXiv's policy on authors' use of generative AI language tools (`https://info.arxiv.org/help/moderation/index.html`).

Code and data availability. The benchmark, all experiment configurations, and the code to reproduce every number in this paper are available at `https://github.com/llvm-x86/fast-weights-predict`. `bench.py` is the numpy-accelerated implementation used to produce all numbers in this paper; `bench.js` is the reference JavaScript implementation.

References

[1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, and M. Bartoszkiewicz. The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507, 2025.

[2] J. Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[3] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[4] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[5] R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.

[6] N. Frémaux and W. Gerstner. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9:85, 2015.

[7] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. arXiv:2301.04104, 2023.

[8] B. Widrow and M. E. Hoff. Adaptive switching circuits. In *IRE WESCON Convention Record*, 1960.

[9] N. A. Shneydor. *Missile Guidance and Pursuit: Kinematics, Dynamics and Control*. Horwood Publishing, 1998.

[10] P. J. Nahin. *Chases and Escapes: The Mathematics of Pursuit and Evasion*. Princeton University Press, 2007.

[11] B. R. Fajen and W. H. Warren. Behavioral dynamics of steering, obstacle avoidance, and route selection. *Journal of Experimental Psychology: Human Perception and Performance*, 29(2):343–362, 2003.

[12] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[13] R. L. Plackett. Some theorems in least squares. *Biometrika*, 37(1/2):149–157, 1950.

[14] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[15] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[16] B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.